

COMP2000

Traffic Simulation

Complete codebase reading and defence guide

A practical explanation of the current Java project: what every package does, how one tick works, how collision avoidance is calculated, how traffic lights and buses behave, and what to say in a project defence.

Use this document beside the repository. Read the files in the suggested order, then follow one vehicle through the execution trace. You do not need to memorise every line; you need to understand the responsibility of each class and the reason behind the important calculations.

1. The project in one minute

The program is a Java Swing traffic simulation. Cars, trucks, buses and emergency vehicles travel straight through a grid of signalised intersections. It does not simulate turns, roundabouts, pedestrians, or physics collisions.

The core is deliberately simple: each tick updates lights, lets every vehicle sense hazards, moves every vehicle safely, removes vehicles that leave the map, possibly spawns another vehicle, then redraws the display.

```
network file -> roads, lanes, intersections, bus stops
      |
      v
each tick: update lights -> read sensors -> move vehicles
              -> despawn -> maybe spawn -> repaint
```

Concern	Main class or file	Plain-language responsibility
Startup	Main.java	Loads the map, sets a run seed, creates the engine and window.
Map construction	NetworkLoader.java + grid.txt	Builds roads, lanes, lights and stops.
Simulation loop	SimulationEngine.java	Runs fixed updates at 30 ticks per second.
Safety	TrafficSensor.java + SensorReading.java	Finds hazards and stopping distances.
Movement	Vehicle.java and subclasses	Changes speed and position.
Signals	TrafficLight.java + state classes	Alternates horizontal and vertical traffic.
Spawning	VehicleSpawner.java	Adds random vehicles on valid boundary lanes.
Display	SimulationDisplay.java	Draws the city and handles Space/R keys.

2. Recommended reading order

Start with the execution path. This prevents getting lost in drawing coordinates before you understand the simulation.

- Main.java - see how the application is assembled.
- networks/grid.txt - see the actual map data.
- NetworkLoader.java - see how text becomes Java objects.
- SimulationEngine.java - learn the tick order.
- TrafficSensor.java and SensorReading.java - learn collision avoidance.
- Vehicle.java, then Car, Truck, Bus and EmergencyVehicle.
- RoadNetwork, Road, Lane, BusStop and intersections.
- TrafficLight and its GreenState, YellowState and RedState classes.
- VehicleSpawner and WeightedRandom.
- SimulationDisplay last - it is presentation, not traffic logic.

For every class, ask three questions: what state does it own? what does it change? who calls it?

3. Startup and the first objects

The entry point is `trafficsim.Main.main(String[] args)`. Java starts there.

3.1 Main.java

Step	What Main does	Why it matters
1	Reads an optional network path and --seed=.	A seed makes one run repeatable.
2	Uses SecureRandom when no seed is supplied.	Normal launches produce different traffic patterns.
3	Calls Vehicle.seedNoise(...).	Driver slowdown has its own deterministic random stream.
4	Loads networks/grid.txt when available.	The default map is data-driven, not hard-coded in the UI.
5	Creates VehicleSpawner and calls spawnOne six times.	The simulation starts with traffic already visible.
6	Creates SimulationEngine(network, 30, spawner, runSeed).	30 means the target update rate is 30 ticks per second.
7	Creates SimulationDisplay inside SwingUtilities.invokeLater.	Swing UI work belongs on Swing's event thread.

```
long runSeed = seed != null ? seed : new SecureRandom().nextLong();
Vehicle.seedNoise(runSeed ^ 0x9E3779B97F4A7C15L);
```

The XOR line derives a second repeatable seed. The spawner and the small random driver slowdowns do not consume exactly the same random sequence, but supplying the same run seed still reproduces the run.

3.2 Running the project

```
# From C:\Users\YAKOV\COMP2000
java src/trafficsim/Main.java

# Repeat one run exactly
java src/trafficsim/Main.java --seed=123
```

The dashboard displays the current run seed in hexadecimal. If something interesting happens during a demonstration, record that seed and launch with it later.

4. The map file and network construction

4.1 grid.txt

grid.txt is a small domain-specific format. It separates map data from Java code.

```
ROAD 55 170 1045 170 50 EAST, WEST
INTERSECTION 250 170 50 30 3
BUS_STOP 430 341 EAST Central_Station
```

Line type	Fields	Meaning
ROAD	x1 y1 x2 y2 speedLimit directions	Creates a straight road and one Lane for every listed direction.
INTERSECTION	x y greenTicks yellowTicks redTicks	Creates a signalised crossing at that coordinate.
BUS_STOP	x y direction name	Creates a named stop on one exact lane.

The current layout contains three east-west roads, three north-south roads and nine signalised crossings. There are no ROUNDABOUT entries because roundabouts were removed.

4.2 NetworkLoader.java

NetworkLoader is the Factory. Its loadFromFile method reads each non-comment line, splits it on whitespace, and uses a switch expression to create the correct Java object. Invalid counts, numbers or directions throw InvalidNetworkException instead of silently building a broken map.

```
String[] parts = line.split("\\s+");
switch (parts[0]) {
```

```

        case "ROAD" -> ...
        case "INTERSECTION" -> ...
        case "BUS_STOP" -> ...
        default -> throw new InvalidNetworkException(...);
    }
}

```

After reading, `wireIntersections` connects an intersection to every road whose centre line passes through its coordinates. Sensors use this connection later to ignore unrelated intersections.

5. Road, lane and bus-stop geometry

5.1 Object relationship

```

RoadNetwork
  contains many Road objects
  contains many SignalisedIntersection objects
  contains many BusStop objects

Road
  contains one Lane per travel direction

Lane
  contains Vehicle objects
  has a back-reference to its Road

```

`RoadNetwork` is the world container. `Road` describes one line segment. `Lane` is the important traffic container: every vehicle belongs to one lane, and therefore has a fixed direction and road.

5.2 Australian left-side traffic

The display uses screen coordinates: `x` increases rightwards and `y` increases downwards. A lane is shifted from the road centre by `Lane.LANE_HALF_WIDTH`, which is 9 world units.

Direction	Movement vector	Left-side lane offset	Meaning on screen
EAST	(1, 0)	(0, -9)	Upper lane of a horizontal road.
WEST	(-1, 0)	(0, +9)	Lower lane of a horizontal road.
SOUTH	(0, 1)	(+9, 0)	Right lane of a vertical road.
NORTH	(0, -1)	(-9, 0)	Left lane of a vertical road.

```

laneCentreX = roadCentreX + direction.leftX() * Lane.LANE_HALF_WIDTH;
laneCentreY = roadCentreY + direction.leftY() * Lane.LANE_HALF_WIDTH;

```

This same formula is used for spawning and for validating bus-stop positions. The University stop is southbound at `x=559` because the vertical road centre is `x=550` and the southbound left-side lane is 9 units to the right.

5.3 BusStop.serves(Lane)

A stop serves a lane only when all three conditions are true: the directions match, the stop sits on that lane's shifted centre line, and it lies between the road endpoints. This prevents a bus from stopping on the wrong side of a road.

6. The simulation engine and one full tick

6.1 SimulationEngine.java

`SimulationEngine` is the coordinator. It owns the `RoadNetwork`, `VehicleSpawner`, tick count, paused state and observer list. It does not draw roads or decide vehicle colours.

```

private void doStep() {
    synchronized (network) {
        tickCount++;
        updateLights();
        List<Vehicle> vehicles = collectVehicles();
    }
}

```

```

        Map<Vehicle, SensorReading> readings = readSensors(vehicles);
        moveLaneVehicles(readings);
        spawner.despawnOffMap(network);
        if (vehicles.size() < SimConstants.SPAWN_CAP) spawner.tick(network);
    }
    notifyObservers();
}

```

Phase	What happens	Why this order is useful
Lights	Every intersection advances one state timer.	Vehicles see the current signal state.
Collect	Engine gathers vehicles from every lane.	Creates one set of active objects for this tick.
Sense	TrafficSensor creates a SensorReading per vehicle.	All drivers decide from the same initial snapshot.
Move	Each vehicle updates speed and position.	Movement uses the precomputed safe distance.
Despawn	Vehicles outside the enlarged map rectangle are removed.	The map does not fill with invisible old vehicles.
Spawn	A new vehicle may appear if below the cap.	Traffic remains populated but bounded.
Notify	SimulationDisplay calls repaint().	The UI redraws only after a complete tick.

6.2 Timing and thread safety

run() starts a daemon thread named sim-loop. It converts tick rate into a time budget. At 30 Hz, one tick has roughly $1,000,000,000 / 30 = 33,333,333$ nanoseconds.

```

long tickNanos = 1_000_000_000L / Math.max(1, tickRate);
nextTick += tickNanos;
long waitNanos = nextTick - System.nanoTime();
if (waitNanos > 0) LockSupport.parkNanos(waitNanos);
else nextTick = System.nanoTime();

```

Both the engine and SimulationDisplay synchronise on the network. That prevents the UI from painting one vehicle before a move and another after a move in the same frame.

7. Vehicles and movement calculations

7.1 Vehicle is the base class

Vehicle is an abstract sealed class. It owns the shared state: x/y position, speed, maxSpeed, physical length, fixed Direction, acceleration step, DriverProfile and current Lane. Only Car, Truck, Bus and EmergencyVehicle may extend it.

Class	Max speed	Length	Special behaviour
Car	3.0	18	Standard movement.
Truck	2.0	28	Cargo weight reduces acceleration.
Bus	2.5	32	Stops once at an assigned stop and picks up passengers.
EmergencyVehicle	4.5	22	Siren-on vehicle ignores red lights but not physical conflicts.

7.2 Vehicle.move(SensorReading)

```

double obstacle = r.effectiveStopDistance() - 1.0;
double target = Math.min(maxSpeed, Math.max(0, obstacle));
double effectiveAccel = accelStep * profile.accelMultiplier();

```

```

if (target < speed) speed = target;
else if (speed < target) speed = Math.min(target, speed + effectiveAccel);

x += direction.dx() * speed;
y += direction.dy() * speed;

```

Expression	Meaning
effectiveStopDistance - 1.0	Keeps a one-unit spare buffer. If a sensor says danger is at zero distance, target speed becomes zero.
min(maxSpeed, ...)	A vehicle cannot drive faster than its own class maximum, even on an empty road.
max(0, obstacle)	Prevents a negative speed when overlap risk is detected.
accelStep * profile multiplier	Aggressive drivers accelerate more quickly; cautious drivers more slowly.
direction vector * speed	Moves straight only. Direction is final, so the vehicle never turns.

Braking is immediate in this simplified model: when target speed is lower than current speed, speed becomes target immediately. Acceleration is gradual. This is conservative and helps prevent overlap when a hazard appears.

7.3 Driver profiles and random slowdown

Profile	Acceleration multiplier	Slowdown probability	Effect
AGGRESSIVE	1.6	0.008	Starts faster and rarely slows randomly.
NORMAL	1.0	0.020	Baseline behaviour.
CAUTIOUS	0.7	0.040	Starts gently and slows more often.

Random slowdown is deliberately small: it creates imperfect driver behaviour and uneven queues without replacing the safety logic. Emergency vehicles override it with a zero probability.

8. Sensors and collision avoidance

8.1 SensorReading

SensorReading is an immutable Java record containing two distances: distToVehicleAhead and distToRedStop. Positive infinity means no relevant obstacle was found. effectiveStopDistance returns the smaller one.

```

public double effectiveStopDistance() {
    return Math.min(distToVehicleAhead, distToRedStop);
}

```

8.2 Same-lane spacing

TrafficSensor.nearestVehicleAhead loops through vehicles in the current lane. It first finds whether another vehicle is ahead, then converts centre-to-centre distance into safe free space between bumpers.

```

centreToCentre = signedDistanceAhead(self, other.x, other.y);
noseToTail = centreToCentre
    - self.length / 2
    - other.length / 2
    - safetyGap;

```

Example: if centres are 30 units apart, the current car is 18 units long, the car ahead is 18 units long and the gap is 4, free space is $30 - 9 - 9 - 4 = 8$ units. The follower limits its speed to fit safely into that space.

8.3 Signed distance: the dot product

```

double dx = pointX - vehicleX;
double dy = pointY - vehicleY;

```

```
return dx * direction.dx() + dy * direction.dy();
```

This is a dot product. For an eastbound vehicle, direction is (1,0), so the result is simply dx: positive is east/ahead and negative is west/behind. For southbound, direction is (0,1), so dy decides. This gives a signed distance along the direction of travel without separate logic for all four directions.

8.4 Intersection stops

Sensor check	What it prevents
nearestSignalStopAhead	Normal cars stop before red or yellow lights.
nearestBlockedIntersection	A car with green does not enter if stopped traffic just beyond the crossing would block the box.
nearestOccupiedIntersectionStop	A vehicle waits if perpendicular traffic is physically inside the junction.
nearestEmergencyPriorityStop	Normal traffic yields to an approaching, clear-path, siren-on emergency vehicle.

The virtual stop line is `STOP_LINE_RADIUS = 52` units before an intersection centre. The junction occupancy radius includes half the other vehicle length, so a vehicle is treated as occupying the intersection until its rear clears it.

8.5 Emergency vehicles

`EmergencyVehicle.stopsAtRedLight` returns `!sirenOn`. With the siren on, the red-light distance is ignored, but same-lane distance and occupied-intersection distance are still checked. This is the exact meaning of emergency priority without collision immunity.

9. Traffic lights and adaptive intersections

9.1 Axis-based lights

`TrafficLight` controls axes rather than individual directions. EAST and WEST are horizontal; NORTH and SOUTH are vertical. One axis owns the active cycle while the perpendicular axis is red.

```
Horizontal GREEN -> Horizontal YELLOW -> ALL RED
-> Vertical GREEN -> Vertical YELLOW -> ALL RED
-> Horizontal GREEN
```

9.2 State pattern

State class	update(...) behaviour
GreenState	When timer reaches <code>greenDuration</code> , set state to <code>YellowState</code> .
YellowState	When timer reaches <code>yellowDuration</code> , set state to <code>RedState</code> .
RedState	When timer reaches <code>redDuration</code> , swap axis and set <code>GreenState</code> .

`TrafficLight` increments timer first, then delegates the transition decision to the current state object. `setState` resets timer to zero. This is the State pattern: each phase knows when it should change.

9.3 Adaptive green extension

`SignalisedIntersection` checks the final four ticks of green. If at least four stopped vehicles are near the intersection on the green axis, it rewinds the timer by 12 ticks. It may do this at most three times in one green phase, ensuring perpendicular traffic still gets a turn.

```
remaining = greenDuration - timer;
if (remaining <= 4 && queued >= 4 && extensions < 3) {
    timer = Math.max(0, timer - 12);
}
```

10. Spawning, randomness and buses

10.1 VehicleSpawner

The spawner uses an injected Random object. Main supplies new Random(runSeed), so normal runs vary while --seed gives repeatability. It has two weighted selections: vehicle type and driver profile.

Vehicle	Weight	Long-run share
Car	75	75%
Truck	12	12%
Bus	6	6%
Emergency vehicle	7	7%

Weights are probabilities over many spawns, not a promise that every short period has exact percentages. Two buses can appear close together, and a station can go a while without a bus, because lanes and types are independently random.

10.2 Boundary-lane selection

For each road lane, laneEntryPoint finds the edge from which that travel direction enters the map. The spawner keeps only points on the network bounding box, shuffles the complete list, and uses the first lane that is not crowded near its entry. Buses are intentionally allowed to spawn on any valid boundary lane, including a route without a stop.

```
if (rng.nextDouble() > spawnProbability) return;
Collections.shuffle(candidates, rng);
for (SpawnPoint candidate : candidates) {
    if (laneIsCrowdedNearEntry(candidate.lane, candidate.entry)) continue;
    // create and add one vehicle
}
```

10.3 WeightedRandom

WeightedRandom adds all weights, picks a random number from zero up to that total, and returns the first cumulative weight bucket containing the number. For example, with Car=75 and Truck=12, rolls from 0 up to 75 select cars; the next 12 select trucks.

10.4 Bus behaviour

When a bus is spawned, it is given every BusStop whose serves(lane) method is true. If it happens to have an assigned stop, it stops within an 8-unit radius, picks up up to three passengers, and dwells for 30 ticks. servedStops prevents repeated stopping at the same stop. If a bus has no matching stop, it remains valid and continues straight through the network.

11. User interface and rendering

11.1 SimulationDisplay

SimulationDisplay extends JPanel and implements SimulationObserver. The engine adds it as an observer; after a complete tick it calls onSimulationStep, which calls repaint. Swing then invokes paintComponent.

```
paintComponent order:
1. drawCityBlocks
2. drawRoads
3. drawBusStops
4. drawIntersections
5. drawVehicles
6. drawDashboard
```

Drawing order is visual layering. Roads must be painted before vehicles, and dashboard last so it remains readable. The renderer never changes vehicle positions; it only reads state from the engine.

11.2 Icons, colours and controls

Feature	Implementation
Car colours	IdentityHashMap gives each exact vehicle object a stable colour for its lifetime.
Bus icon	Long yellow rounded rectangle with repeated windows.
Truck icon	Purple body with a pale cargo panel.
Emergency icon	White vehicle with red and blue light bar when siren is on.
Signals	A green, yellow or red circular bulb drawn from <code>phaseFor(direction)</code> .
Space	Toggles paused state.
R	Clears vehicles and resets tick count; normal spawning then repopulates the map.

The University bus shelter is rendered on the right of the southbound vertical lane. This matches Australian left-side lane placement on a screen where y increases downwards.

12. Every package and class at a glance

Package	Classes	Role
trafficsim	Main, SimConstants	Entry point and shared tuning values.
engine	SimulationEngine, TrafficSensor, SensorReading, Statistics, SimulationObserver, VehicleSpawner	Tick loop, safety readings, metrics, updates and spawning.
factory	NetworkLoader	Reads file data into a RoadNetwork.
model.road	RoadNetwork, Road, Lane, Intersection, SignalisedIntersection, BusStop	Map structure and road behaviour.
model.light	TrafficLight, LightState, GreenState, YellowState, RedState	Signal state machine.
model.vehicle	Vehicle, Car, Truck, Bus, EmergencyVehicle, DriverProfile	Traffic actors and movement differences.
strategy	WeightedRandom	Reusable generic weighted choice.
util	Direction, Axis, LightPhase	Small enums and geometry helpers.
view	SimulationDisplay	Swing rendering and keyboard input.
exception	SimulationException, InvalidNetworkException	Clear errors for invalid map data.

13. Patterns and Java features

Concept	Where it appears	Why it is useful here
Inheritance and polymorphism	Vehicle -> Car/Truck/Bus/EmergencyVehicle	Engine can store every vehicle as Vehicle and call move without knowing its exact subtype.
Sealed classes	Vehicle and Intersection	Compiler limits allowed subclasses and makes the domain explicit.

Concept	Where it appears	Why it is useful here
State pattern	TrafficLight and the three state classes	Keeps phase transition rules out of one giant conditional.
Factory pattern	NetworkLoader	Builds a complete network from text data or fallback defaults.
Observer pattern	SimulationObserver and SimulationDisplay	Engine informs UI after an update without depending on drawing details.
Generics	WeightedRandom, List	Type-safe reusable collections and weighted selection.
Record	SensorReading	Compact immutable transfer object for per-tick sensor data.
Streams/lambda	Statistics and NetworkLoader	Concise collection processing where it improves readability.

14. Trace one car through a tick

Use this trace when explaining the system verbally.

Moment	What happens
1. Car exists	It sits in an eastbound Lane with a position, speed, length and driver profile.
2. Engine updates lights	The car's next intersection may become yellow or remain green.
3. Sensor reads	TrafficSensor finds the closest same-lane vehicle and closest required intersection stop.
4. Reading returned	SensorReading stores those two distances. It does not change the world.
5. Vehicle moves	Vehicle.move chooses the lower safe target speed, brakes immediately if needed, otherwise accelerates gradually.
6. Position changes	x increases by speed because an eastbound direction vector is (1,0). y remains unchanged.
7. UI repaints	The display draws the new position after the complete tick is finished.

15. Defence questions and answers

How do you prevent collisions?

Each tick, every vehicle receives a sensor snapshot. It limits speed by the closest same-lane free space or intersection stop. Vehicles also wait for occupied intersections and blocked downstream lanes.

Do emergency vehicles ignore all safety rules?

No. A siren-on emergency vehicle ignores red lights only. It still respects same-lane spacing and waits for occupied or conflicting intersections.

Why do vehicles never turn?

The simplified requirement is straight-through traffic. Direction is fixed in Vehicle, and movement only adds direction vector times speed. There is no lane-transfer or direction-change code.

Why is the simulation random but repeatable?

Normal launches use SecureRandom to make a seed. Passing --seed supplies the same seed, so spawning and driver slowdown repeat deterministically.

What is a tick?

One complete simulation update. At the target 30 Hz, it is roughly 33 milliseconds of simulated update scheduling.

Why use a State pattern for signals?

Green, yellow and red have different transition rules. Separate state objects make each rule small and avoid a large phase conditional.

Why use synchronisation?

The simulation and Swing drawing run on different threads. Locking the network makes a frame show a complete state rather than a partial update.

Why do buses sometimes not stop?

Buses may spawn on any valid boundary lane. They only receive a stop if their exact lane serves a configured stop; otherwise they continue straight, which preserves full spawn randomness.

16. Important constants to know

Constant	Value	Meaning
LANE_WIDTH / LANE_HALF_WIDTH	18 / 9	Two directions occupy one 36-unit-wide road; each lane centre is 9 units from road centre.
STOP_LINE_RADIUS	52	Normal stop point before a junction centre.
SIGHT_RANGE	250	Hazards farther ahead are ignored for this tick.
MIN BUMPER_GAP	4	Extra free space after subtracting vehicle half-lengths.
SPAWN_CAP	18	Maximum active vehicles before automatic spawning pauses.
SPAWN_PROBABILITY	0.06	Chance per tick of attempting one spawn below the cap.
DWELL_TICKS	30	How long a bus waits at a served stop.
ADAPTIVE_QUEUE_HIGH	4	Stopped vehicles needed before green may extend.

17. Final checklist before your defence

- Run the program once normally and once with --seed=123.
- Point at the dashboard seed and explain reproducibility.
- Follow one car through SimulationEngine -> TrafficSensor -> Vehicle.move.
- Explain the centre-distance to bumper-gap calculation in your own words.
- Show a red light, then explain STOP_LINE_RADIUS and the all-red phase.
- Show a bus stop and explain why a bus only serves it when its lane matches.
- Show an emergency vehicle and state: priority, not collision immunity.
- Know that the display reads state; it does not control traffic movement.
- Be honest about scope: no turns, roundabouts, pedestrians or physics engine.